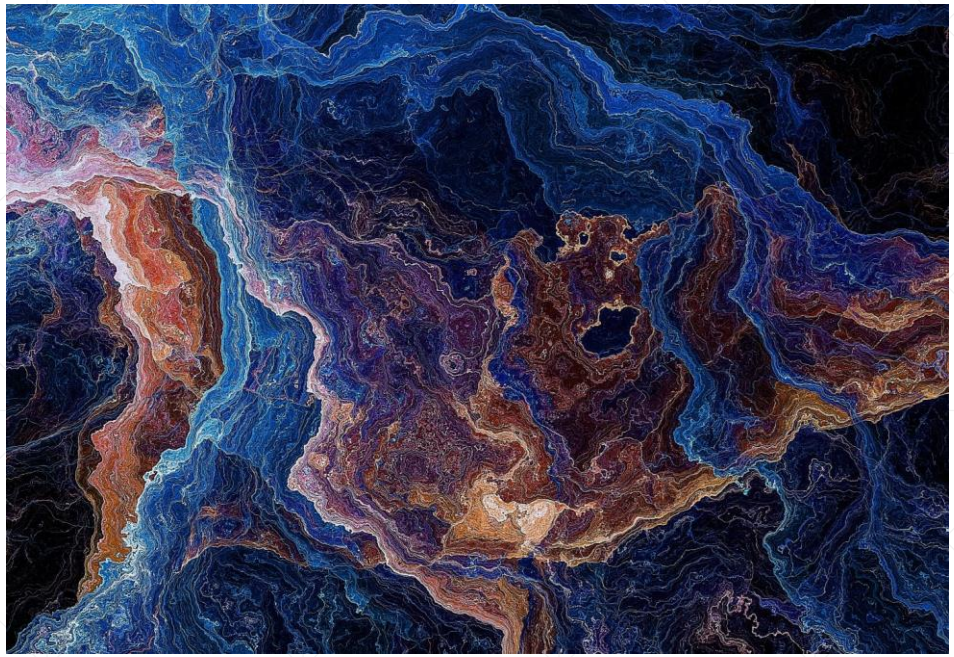
Java网络开发实战之



网络设备搜索

北京理工大学计算机学院
金旭亮

应用背景及技术方案

概述

在真实的网络环境中，经常发生特定网络设备的上线与退出情况，因此，控制中心能维护一个“当前在线”的网络设备清单，是非常重要的。

在本讲中，我们将针对一个“最简单”的网络环境—同一个局域网，展示如何动态地搜索局域网中在线的网络设备。

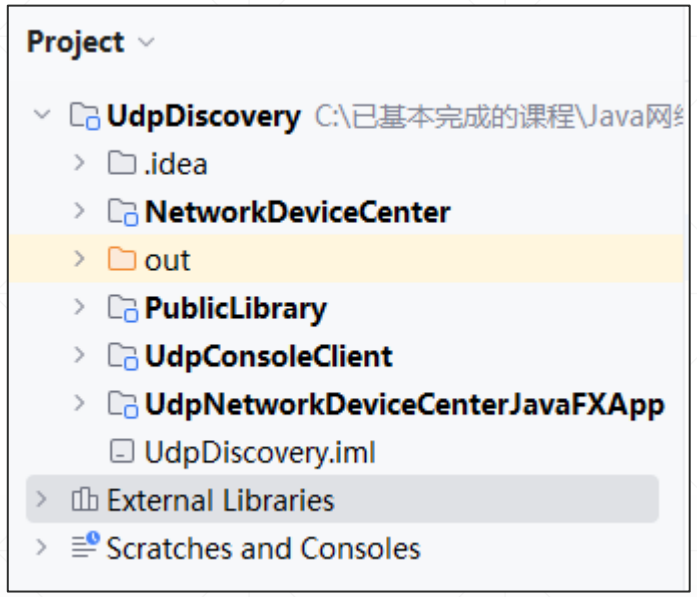
基本思路

1. 利用UDP的广播特性，控制中心应用定期向网络中广播消息，其中包容了联系控制中心所需的所有相关信息。

2. 各上线的网络设备，按照事先的约定，从指定的端口中接收广播信息，收到之后解析出相关信息，然后，向控制中心汇报“自己在线或已上线”。

3. 控制中心接收到各网络设备发来的“报告”之后，进行汇总，使用一张表展示出“当前在线”的设备清单。

示例解决方案



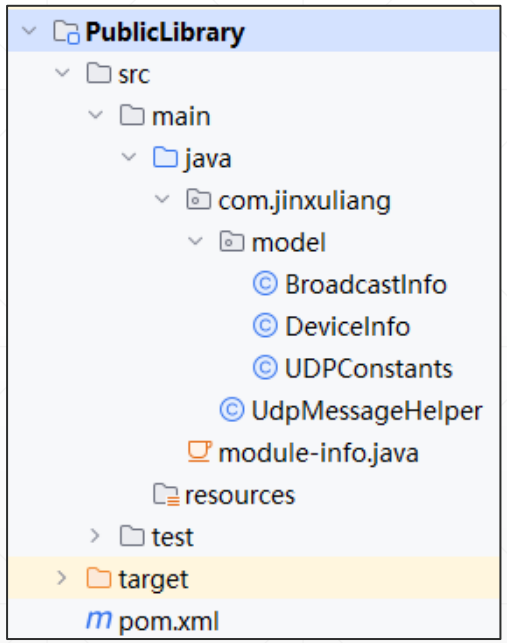
示例项目由以下几个模块组成：

- PublicLibrary模块：保存一些公用代码。
- UdpConsoleClient模块：一个控制台程序，代表一个网络设备
- NetworkDeviceCenter模块：一个控制台程序，它模拟设备控制中心的基础功能，主要用于开发和测试
- NetworkDeviceCenterJavaFX模块：一个JavaFX程序，它提供了设备控制中心的完备功能，这个是真正的设备控制中心。

公用模块

PublicLibrary模块的职责

这个模块，主要是封装了那些整个项目中都需要用到的数据类，以及实现UDP信息收发的一个Helper类型。




```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
</dependency>

<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.30</version>
</dependency>
```

为了简化代码编写，引入了Lombok，同时，引入Gson完成Json序列化与反序列化的功能。

PublicLibrary是一个JPMS模块

由于JavaFx应用已经模块化了，为了方便它调用PublicLbrary模块中的代码，我们也将其设置为一个JPMS模块。

module-info.java (PublicLibrary)

```
1  module PublicLibrary {
2      exports com.jinxuliang;
3      requires lombok;
4      opens com.jinxuliang;
5  }
```

消息相关类型

控制中心广播的消息

```java
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class BroadcastInfo {
    //设备控制中心IP地址
    private String deviceCenterIp;
    //设备控制中心监听的端口（当前仅用于UDP）
    private int deviceCenterListenPort;
}
```

网络设备的汇报消息

```java
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
@EqualsAndHashCode
public class DeviceInfo {
    //网络设备IP
    private String ip;
    //网络设备监听的端口（用于接收信息）
    private int port;
    //网络设备信息
    private String info;
}
```

相关常量

```java
public class UDPConstants {
    //各个设备在此端口监听
    6 usages
    public static int DEVICE_LISTEN_PORT = 20000;
    //这个是设备中心监听的端口
    5 usages
    public static int MANAGER_LISTEN_PORT = 30000;
}
```

每个网络设备，应该监听同一个端口，以便接收到广播消息，同时，设备控制中心也应该有一个端口，用于接收设备发来的“汇报”信息。

封装UDP网络数据发消息功能

```
//将指定的消息发送给指定IP和端口的目标计算机
//最后一个参数可选，不为空时，将在控制台窗口中显示它
3 usages
public static void sendMessageTo(String message, String destinationIP,
                                 int destinationPort,String statusInfo)
        throws IOException {
    byte[] messageData = message.getBytes(StandardCharsets.UTF_8);
    //目标地址是必须设置的，否则，会抛出异常: IllegalArgumentException
    var address = InetAddress.getByName(destinationIP);
    var sendPacket = new DatagramPacket(messageData, messageData.length, address,
            destinationPort);
    try (var datagramSocket = new DatagramSocket()) {
        datagramSocket.send(sendPacket);
        if(!statusInfo.isEmpty()){
            System.out.println(statusInfo);
        }
    }
}
```

封装UDP网络数据接收消息功能

```
//从DatagramSocket中接收信息，以字符串方式返回收到的信息
//第2个参数为true时，在控制台显示相关信息
3 usages
public static String  receiveMessage(DatagramSocket ds,boolean showInfo) throws IOException {
        var dataBuffer = new byte[1024];
        var receivePack = new DatagramPacket(dataBuffer, dataBuffer.length);
        //在此阻塞等待接收
        ds.receive(receivePack);
        String ip = receivePack.getAddress().getHostAddress();
        int port = receivePack.getPort();// 发送者的端口
        int dataLen = receivePack.getLength(); //获取接收到的数据长度
        //转换为字符串
        String message = new String(receivePack.getData(), offset: 0, dataLen);
        if(showInfo){
            System.out.println("\n收到外部发来的信息: " + message);
            System.out.print("外部数据包的");
            System.out.println("ip:" + ip + "\tport:" + port);
        }
        return message;

}
```

网络设备端应用

```java
public class ConsoleClientMain {
    //使用UUID作为网络设置标识
    2 usages
    private static final String CLIENT_ID = UUID.randomUUID().toString();

    public static void main(String[] args) throws IOException {
        var discoverClientThread = new Thread(() -> {
            while (true) {
                System.out.println("\nUDP客户端:" + CLIENT_ID + "在端口" +
                        UDPConstants.DEVICE_LISTEN_PORT + "监听");
                try {
                    //接收广播消息，收到之后，回发“汇报”消息给控制中心
                    waitingDiscoveryMessage();
                } catch (IOException e) {
                    throw new RuntimeException(e);
                }
            }
        });
        discoverClientThread.start();
        System.out.println("敲回车键退出……");
        new Scanner(System.in).nextLine();
        System.exit( status: 0);
    }
```

```
1 usage
private static void waitingDiscoveryMessage() throws IOException {
    try (var ds = new DatagramSocket(UDPConstants.DEVICE_LISTEN_PORT)) {
        //接收广播消息
        var message = UdpMessageHelper.receiveMessage(ds, showInfo: true);
        //解析广播消息
        var deviceCenterInfo = new Gson().fromJson(message, BroadcastInfo.class);
        var myIP = InetAddress.getLocalHost().getHostAddress();
        //生成汇报消息
        var deviceInfo = DeviceInfo.builder().ip(myIP)
                .info(CLIENT_ID).port(UDPConstants.DEVICE_LISTEN_PORT)
                .build();
        Gson gson = new Gson();
        var json = gson.toJson(deviceInfo);
        //发送汇报消息
        UdpMessageHelper.sendMessageTo(json,
                deviceCenterInfo.getDeviceCenterIp(),
                deviceCenterInfo.getDeviceCenterListenPort(),
                statusInfo: "本机信息已经发送到设备控制中心。");
    }
}
```

网络控制中心

（控制台版）

发送广播消息

```
1 usage
private static void sendMessage(String message) throws IOException {
    //设置为广播地址
    UdpMessageHelper.sendMessageTo(message,
            destinationIP: "255.255.255.255",
            UDPConstants.DEVICE_LISTEN_PORT,
            statusInfo: "消息已经发送到目标机器的端口: " + UDPConstants.DEVICE_LISTEN_PORT);
}
```

接收设备发来的汇报信息

```
1 usage
private static void receiveMessage() throws IOException {
    try (var ds = new DatagramSocket(UDPConstants.MANAGER_LISTEN_PORT)) {
        while (true) {
            System.out.println("\n正在等待设备发回响应消息");
            UdpMessageHelper.receiveMessage(ds, showInfo: true);
        }
    }
}
```

由于会有多个设备发来消息，所以，这里采用了循环接收的方式。

在独立的线程中等待设备发来消息

```
public class DeviceCenterMain {
    public static void main(String[] args) throws IOException {
        System.out.println("网络设备控制中心");
        var address = InetAddress.getLocalHost();
        var deviceCenterInfo = BroadcastInfo.builder()
                .deviceCenterIp(address.getHostAddress())
                .deviceCenterListenPort(UDPConstants.MANAGER_LISTEN_PORT);
        var json = new Gson().toJson(deviceCenterInfo);
        Thread thread = new Thread(() -> {
            try {
                receiveMessage();
            } catch (IOException e) {
                System.out.println(e.getMessage());
            }
        });
        thread.start();
```

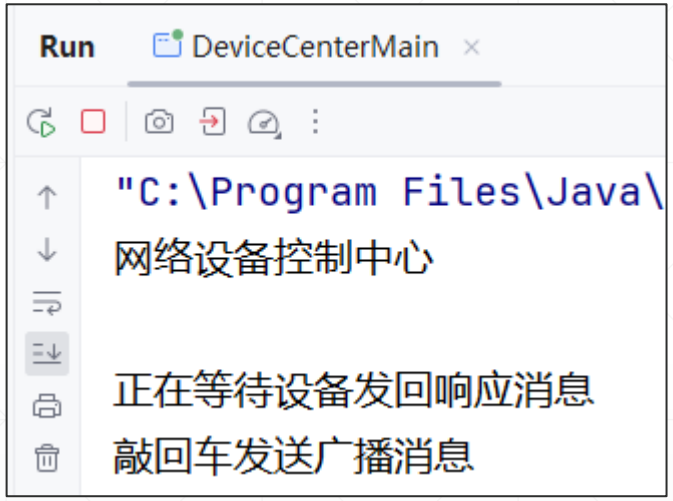
发送广播消息

使用回车，发布多条消息……

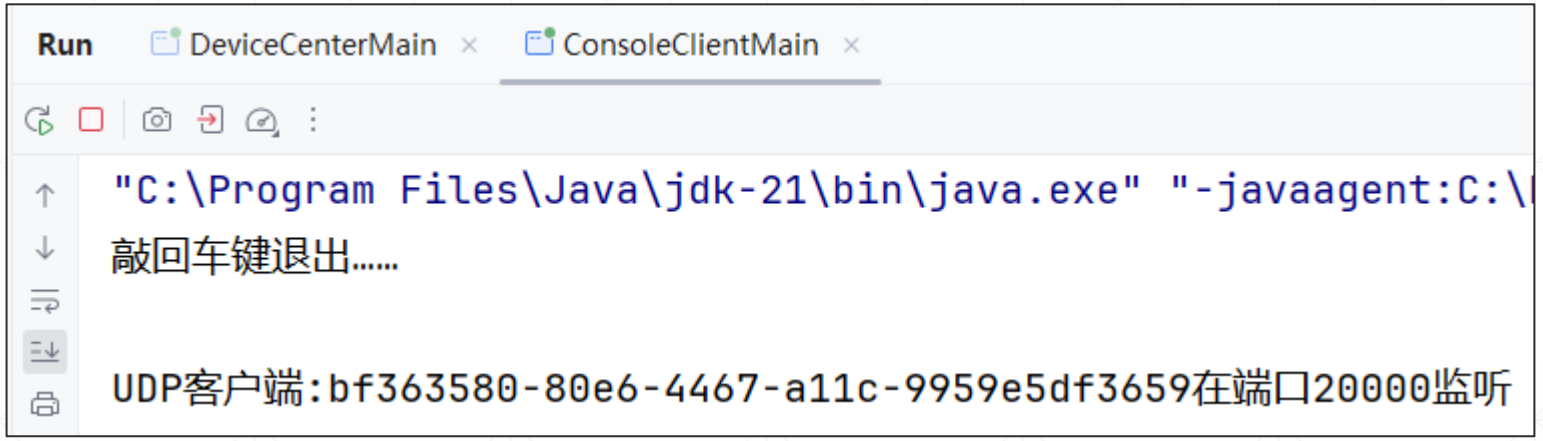
```
    var scanner = new Scanner(System.in);
    System.out.println("敲回车发送广播消息");
    var inputString = scanner.nextLine();
    while (!inputString.equalsIgnoreCase( anotherString: "quit")) {
        sendMessage(json);
        System.out.println("广播消息已经发送，敲回车键继续发送，quit退出");
        inputString = scanner.nextLine();
    }
    System.exit( status: 0);
}
```

测试截图-1

启动控制中心

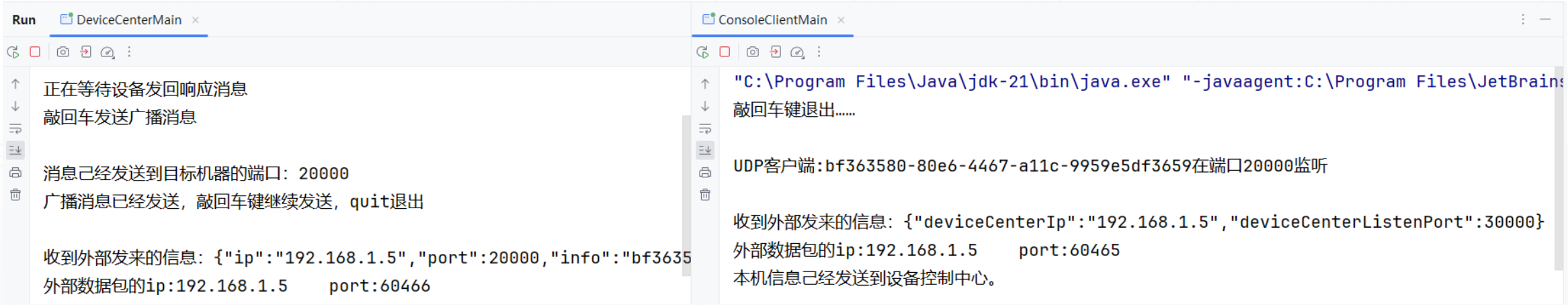


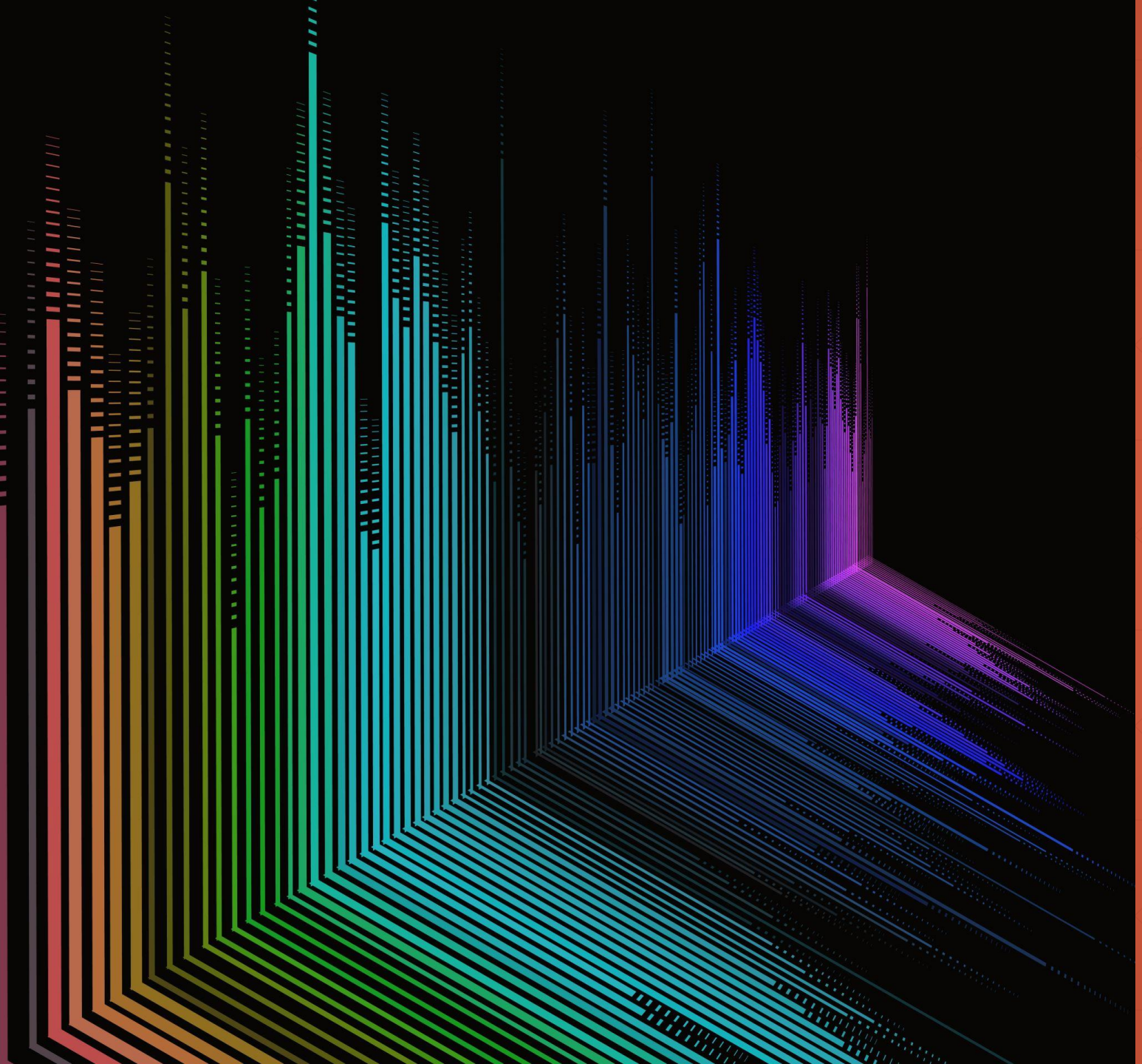
启动一个设备



测试截图-2

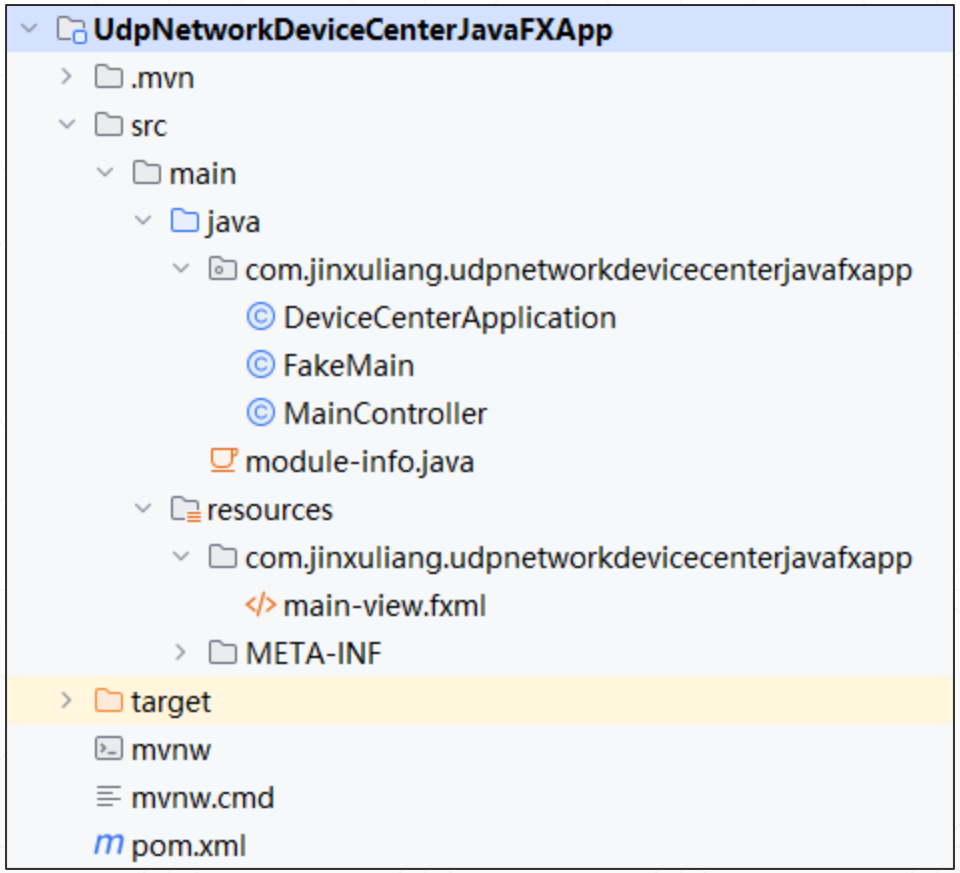
设备中心控制台，敲回车键发送广播消息，可以看到，设备端收到了消息，之后又将本机信息发给了设备中心……





改造控制中心为JavaFX应用

项目结构及依赖



引入了Lombok、RxJava和Gson这些第三方组件。

```xml
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.30</version>
</dependency>
<dependency>
    <groupId>io.reactivex.rxjava3</groupId>
    <artifactId>rxjava</artifactId>
    <version>3.1.6</version>
</dependency>
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
</dependency>
<dependency>
    <groupId>com.jinxuliang</groupId>
    <artifactId>PublicLibrary</artifactId>
    <version>1.0-SNAPSHOT</version>
    <scope>compile</scope>
</dependency>
```

JPMS模块化配置

module-info.java (com.jinxuliang.udpnetworkdevicecenterjavafxapp)

```
1  module com.jinxuliang.udpnetworkdevicecenterjavafxapp {
2      requires javafx.controls;
3      requires javafx.fxml;
4      requires PublicLibrary;
5      requires lombok;
6      requires com.google.gson;
7      requires io.reactivex.rxjava3;
8
9      opens com.jinxuliang.udpnetworkdevicecenterjavafxapp to javafx.fxml;
10     exports com.jinxuliang.udpnetworkdevicecenterjavafxapp;
11 }
```

界面设计


Document
Hierarchy
AnchorPane
Button 发送广播消息
TableView
TableColumn IP地址
TableColumn 端口
TableColumn 信息
Label 点击按钮，通知所有设备报告信息


发送广播消息
点击按钮，通知所有设备报告信息
IP地址
端口
信息
表中无内容

控制器功能详解

```
public class MainController implements Initializable {
    //region "界面控件相关字段"
    @FXML
    private Button btnSend;
    @FXML
    private TableView<DeviceInfo> tableDevices;
    @FXML
    private TableColumn<DeviceInfo, String> ip;
    @FXML
    private TableColumn<DeviceInfo, Integer> port;
    @FXML
    private TableColumn<DeviceInfo, String> info;
    @FXML
    private Label lblInfo;
    //endregion
```

```
//用于保存在线设备的列表
3 usages
private ObservableList<DeviceInfo> deviceInfos =
        FXCollections.observableList(new ArrayList<>());
//用于实现UDP数据的发送与接收
2 usages
DatagramSocket ds = null;
//当主窗体关闭时，结束后台线程
3 usages
Disposable schedularTask = null;
```

控制器初始化过程

```java
@SneakyThrows
@Override
public void initialize(URL url, ResourceBundle resourceBundle) {
    //设定TableView绑定"在线设备集合"
    ip.setCellValueFactory(new PropertyValueFactory<>( s: "ip"));
    port.setCellValueFactory(new PropertyValueFactory<>( s: "port"));
    info.setCellValueFactory(new PropertyValueFactory<>( s: "info"));
    tableDevices.setItems(deviceInfos);

    ds = new DatagramSocket(UDPConstants.MANAGER_LISTEN_PORT);

    btnSend.setOnAction(e -> {
        //点击按钮，立即发送广播消息
        sendBroadcastMessage();
    });
    //使用RxJava，每隔5秒广播一次消息
    schedularTask = Observable.interval( initialDelay: 0,  period: 5, TimeUnit.SECONDS)
            .subscribe((counter) -> {
                showInfo(LocalTime.now() + ": 广播消息……");
                sendBroadcastMessage();
            });
    //等待设备"汇报消息"
    waitForResponse();
}
```

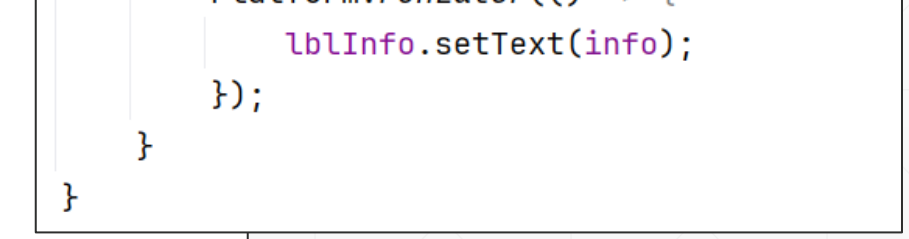
广播消息的发送

```
2 usages
private void sendBroadcastMessage() {
    new Thread(() -> {
        InetAddress address = null;
        try {
            address = InetAddress.getLocalHost();
            var deviceCenterInfo = BroadcastInfo.builder()
                    .deviceCenterIp(address.getHostAddress())
                    .deviceCenterListenPort(UDPConstants.MANAGER_LISTEN_PORT);
            var json = new Gson().toJson(deviceCenterInfo);
            sendMessage(json);
        } catch (IOException ex) {
            showInfo(ex.getMessage());
        }
    }).start();
}
```

```
//使用UDP发送消息
1 usage
private void sendMessage(String message)
        throws IOException {
    //设置为广播地址
    UdpMessageHelper.sendMessageTo(message,
            destinationIP: "255.255.255.255",
            UDPConstants.DEVICE_LISTEN_PORT,
            statusInfo: "");
}
```

等待网络设备回应

```
1 usage
private void waitForResponse() {
    var thread = new Thread(() -> {
        try {
            while (true) {
                //收到设备回发的信息之后，将其加入到在线设备集合中
                var json = UdpMessageHelper.receiveMessage(ds, showInfo: false);
                var deviceInfo = new Gson().fromJson(json, DeviceInfo.class);
                //重复的不再加入集合
                if (!deviceInfos.contains(deviceInfo)) {
                    deviceInfos.add(deviceInfo);
                }
            }
        } catch (IOException e) {
            showInfo(e.getMessage());
        }
    });
    thread.setDaemon(true); //设置为背景线程，以便自动终结
    thread.start();
}
```



```
//可以跨线程安全调用的信息显示方法，使用标签显示信息
3 usages
private void showInfo(String info) {
    if (!info.isEmpty()) {
        Platform.runLater(() -> {
            lblInfo.setText(info);
        });
    }
}
```

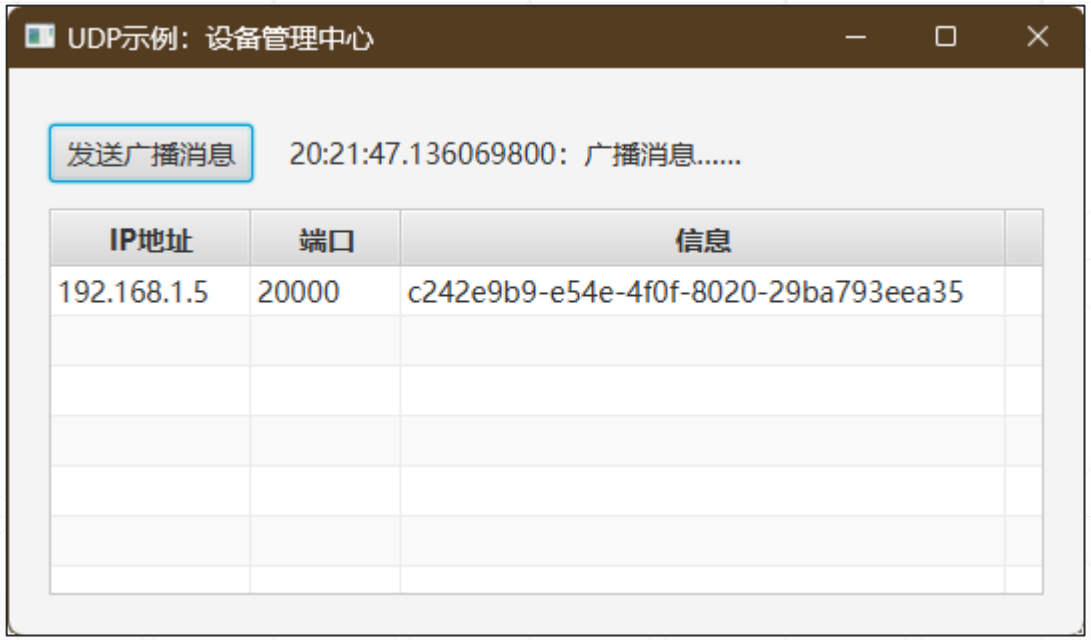
关闭后台线程

控制器中：

```
public void close() {
    if (schedularTask != null) {
        schedularTask.dispose();
    }
}
```

```
public class DeviceCenterApplication extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(
                DeviceCenterApplication.class.getResource(
                        name: "main-view.fxml"));
        Parent root = fxmlLoader.load();
        Scene scene = new Scene(root);
        //获取控制器引用
        MainController controller = fxmlLoader.getController();
        stage.setTitle("UDP示例：设备管理中心");
        stage.setScene(scene);
        //当主窗体关闭时，退出后台线程
        stage.setOnCloseRequest(e -> {
            controller.close();
        });
        stage.show();
    }
```

运行截图



控制中心每隔5秒发一条广播消息，设备端进行响应。

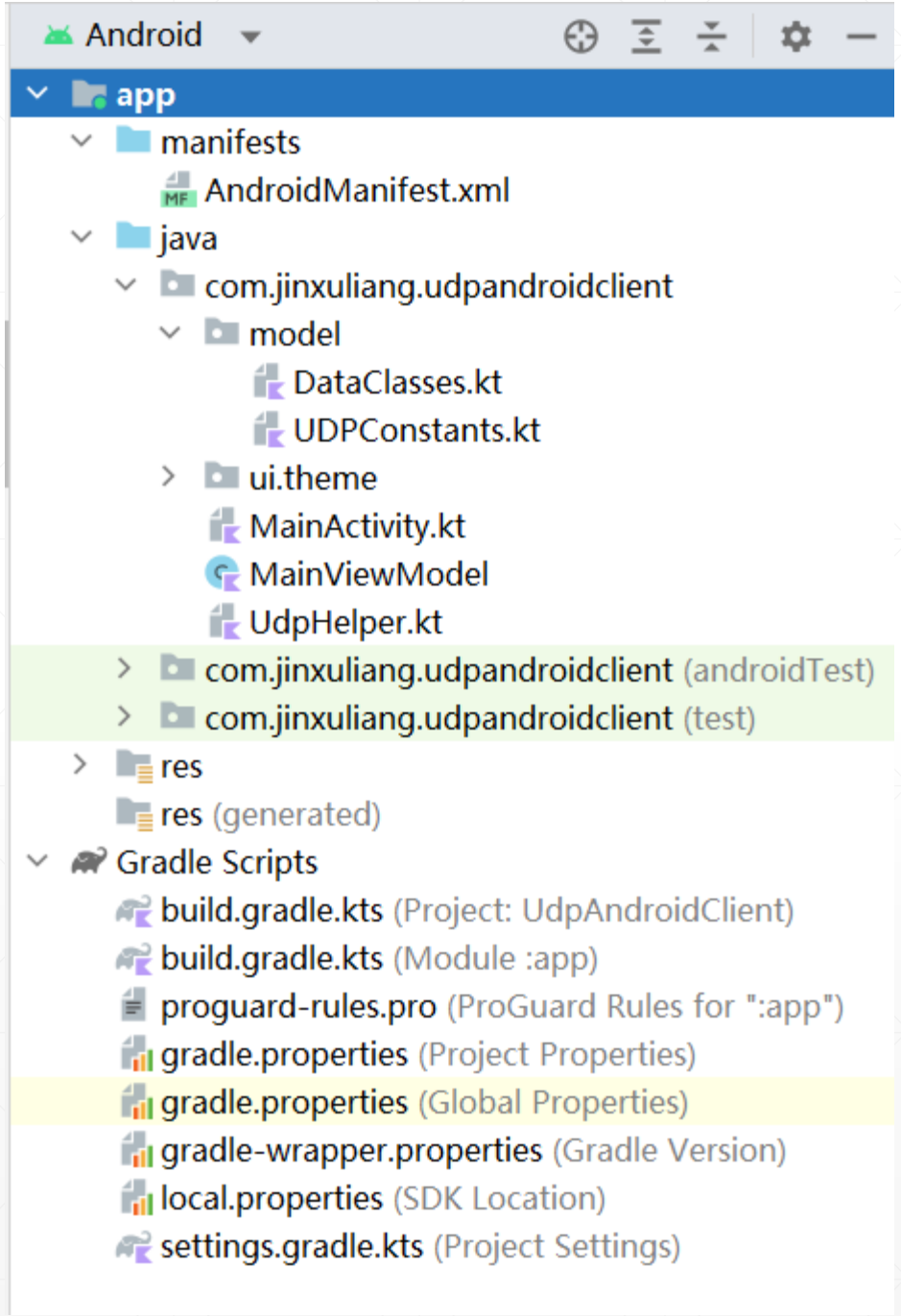
开发手机端App

概述

Android手机可以使用JDK中的各种组件，因此，我们可以写一个Android App，同样使用UDP相关组件来接收和发送信息。

Android原生App开发使用kotlin，本示例使用了协程来实现后台收发网络消息的功能。

本示例使用Jepack Compose来开发UI。



相关数据类

```
//接收到的广播信息
data class BroadcastInfo (val deviceCenterIp:String,
                          val deviceCenterListenPort:Int)

//本手机设备的信息
data class DeviceInfo(
    val ip:String,
    val port:Int,
    val info:String
)

//手机App的IP地址
data class IpInfo(
    val name: String,
    val ip: String
)
```

```
//本手机UDP监听端口
const val DEVICE_LISTEN_PORT = 20000
//本手机设备标识
val CLIENT_ID = UUID.randomUUID().toString()
```

获取本手机的IP地址

```
fun getLocalIpAddress(): String? {
    try {
        val en = NetworkInterface.getNetworkInterfaces()
        while (en.hasMoreElements()) {
            val intf: NetworkInterface = en.nextElement()
            val enumIpAddr = intf.getInetAddresses()
            while (enumIpAddr.hasMoreElements()) {
                val inetAddress: InetAddress = enumIpAddr.nextElement()
                if (!inetAddress.isLoopbackAddress && inetAddress is Inet4Address) {
                    return inetAddress.hostAddress
                }
            }
        }
    } catch (ex: SocketException) {
        Log.d( tag: "IP", ex.message!!)
    }
    return null
}
```

```kotlin
//使用UDP发送消息
fun sendMessageTo(
    message: String, destinationIP: String,
    destinationPort: Int
) {
    val messageData = message.toByteArray(StandardCharsets.UTF_8)
    val address = InetAddress.getByName(destinationIP)
    val sendPacket = DatagramPacket(
        messageData, messageData.size, address,
        destinationPort
    )
    DatagramSocket().use { datagramSocket ->
        datagramSocket.send(sendPacket)
    }
}
```

```kotlin
//从DatagramSocket中接收信息，以字符串方式返回收到的信息
fun receiveMessage(ds: DatagramSocket): String? {
    val dataBuffer = ByteArray( size: 1024)
    val receivePack = DatagramPacket(dataBuffer, dataBuffer.size)
    //在此阻塞等待接收
    ds.receive(receivePack)
    val ip = receivePack.address.hostAddress
    val port = receivePack.port // 发送者的端口
    val dataLen = receivePack.length //获取接收到的数据长度
    //转换为字符串
    val message = String(receivePack.data, offset: 0, length = dataLen)
    println("\n收到外部发来的信息: $message")
    print("外部数据包的")
    println("ip:$ip\tport:$port")
    return message
}
```

ViewModel-1

```
class MainViewModel : ViewModel() {
    //供UI绑定
    private val _info = mutableStateOf( value: "");
    val info: State<String>
        get() = _info

    //等待接收消息
    fun waitForReceiveMessage() {...}

    fun showIPInfo() {
        viewModelScope.launch(Dispatchers.Default) { this: CoroutineScope
            _info.value = getLocalIpAddress()!!
        }
    }
}
```

ViewModel-2

```
//等待接收消息
fun waitForReceiveMessage() {
    viewModelScope.launch(Dispatchers.IO) { this: CoroutineScope
        _info.value = "UDP客户端:$CLIENT_ID 在端口 $DEVICE_LISTEN_PORT 监听"
        val ip = getLocalIpAddress()!!
        while (true) {
            val ds = DatagramSocket(DEVICE_LISTEN_PORT).use { it: DatagramSocket
                val msg = receiveMessage(it)
                val broadcastInfo = Gson().fromJson(msg, BroadcastInfo::class.java)
                //取回数据
                _info.value = broadcastInfo.toString()
                //回发数据
                val deviceInfo = DeviceInfo(ip, DEVICE_LISTEN_PORT, CLIENT_ID)
                val json = Gson().toJson(deviceInfo)
                sendMessageTo(
                    json, broadcastInfo.deviceCenterIp, broadcastInfo.deviceCenterListenPort
                )
                _info.value = "本机信息已经发送到设备控制中心。"
            }
            //等5秒再更新
            delay( timeMillis: 5000)
        }
    }
}
```


```
@Composable
fun MyApp() {
    val vm: MainViewModel = viewModel()
    var info by remember {
        mutableStateOf( value: "")
    }
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(20.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Top
    ) { this: ColumnScope
        Button(onClick = {
            vm.waitForReceiveMessage()
        }) { this: RowScope
            Text( text: "Click me!", fontSize = 30.sp)
        }
        Text(
            vm.info.value, modifier = Modifier.padding(10.dp),
            fontSize = 12.sp
        )
    }
}
```

手机与控制台客户端，均成功识别


```
I  收到外部发来的信息: {"deviceCenterIp":"192.168.1.5","deviceCenterListenPort":30000}
I  外部数据包的ip:192.168.1.5     port:60513
D  tagSocket(72) with statsTag=0xffffffff, statsUid=-1
D  tagSocket(6) with statsTag=0xffffffff, statsUid=-1
```